

Real-Time Acceleration for Medical Image Processing

A Real-Time Breast Ultrasound Classification Pipeline on NVIDIA Holoscan SDK

MathWorks MATLAB-Simulink Challenge, Project 259

Developed in collaboration with Analog Devices, Inc. (ADI)

Rohith Ram V

Student ID: 24BCE0543

B.Tech Computer Science and Engineering

Vellore Institute of Technology, Vellore

June 2026

Table of Contents

Table of Contents	2
1. Abstract	5
2. Introduction	6
2.1 Problem Statement	6
2.2 Motivation and Clinical Relevance	6
2.3 Objectives and Scope	6
3. Background	8
3.1 Ultrasound Image Formation and Beamforming	8
3.2 Deep Learning for Breast Lesion Classification	8
3.3 NVIDIA Holoscan SDK for Real-Time Medical Imaging	9
4. System Architecture	10
4.1 End-to-End Pipeline Overview	10
4.2 Hardware and Software Stack	10
4.3 Development Workflow: MATLAB Prototyping to Holoscan Deployment	10
5. Methodology	12
5.1 Datasets	12
5.2 OASBUD Beamforming Approach	12
5.3 Image Enhancement	14
5.4 CNN Model Development: From BUSI to the Mega Model	15
5.5 ONNX Export and Holoscan Operator Design	16
5.6 Simulink Representation	16
6. Results	18
6.1 Classification Performance	18
6.1.1 Stage-wise Confusion Matrices	18
6.1.2 Final Mega Model Metrics	22
6.2 Real-Time Throughput	26
6.3 Plane-Wave Compounding Study (CIRS040GSE Phantom)	28
6.4 Simulink Representation	29
7. Challenges and Solutions	30
8. Limitations	32
9. Future Work	34
10. Conclusion	35
11. GitHub Repository Guide	36

11.1 Development History	36
11.2 Public Repository Structure and Usage	36
12. References.....	38
13. Appendix	39
13.1 OASBUD Beamforming, Corrected Approach	39
13.2 Holoscan Inference Operator, Preprocessing Convention	39

Executive Summary

Problem	Breast cancer is the most frequently diagnosed cancer worldwide. Ultrasound is a safe, accessible imaging modality but real-time AI-assisted classification of raw RF data remains an open challenge, requiring GPU-accelerated processing to achieve clinically useful frame rates.
Proposed Solution	A six-stage real-time pipeline: raw OASBUD RF data beamformed via DAS (GPU Coder-generated CUDA kernel), log-compressed via Hilbert envelope detection, enhanced with a 3x3 median filter (validated in functional Simulink subsystem), classified by MobileNetV2 CNN trained on 4,429 images across five datasets, and deployed via NVIDIA Holoscan SDK.
Tools Used	MATLAB R2024b (Deep Learning Toolbox, GPU Coder, Image Processing Toolbox, Simulink), Python 3.10, NVIDIA Holoscan SDK 4.2.0, ONNX Runtime GPU, CUDA 12.6, VS2022, RTX 4070 Laptop GPU (8 GB VRAM, Compute Capability 8.9).
Key Innovation	GPU Coder-generated CUDA kernel parallelises 60,000 pixels simultaneously. Combined five-dataset training corpus (4,429 images) improves generalisation. Functional Simulink subsystem validates image processing stages. GreedyScheduler Holoscan pipeline achieves 34 fps end-to-end on RTX 4070 Laptop GPU.
Final Results	87.37% offline test accuracy (combined 5-dataset held-out set). 73% live pipeline accuracy (100 OASBUD RF frames). ~34 fps throughput, ~29 ms average latency. Malignant recall 93.5%.
Limitations	Domain gap between BUSI-trained model and raw plane-wave RF reconstructions reduces live accuracy to 73% (~58% on authentic held-out OASBUD patients). Single plane-wave beamforming limits image quality.
Repository	github.com/rohithgreninja-afk/realtime-ultrasound-holoscan

1. Abstract

This report documents the design, implementation, and evaluation of a real-time breast ultrasound classification pipeline developed for the MathWorks MATLAB-Simulink Challenge (Project 259), undertaken in collaboration with Analog Devices, Inc. The project addresses the gap between offline diagnostic accuracy and the real-time speed required for practical deployment in portable ultrasound systems. Raw radio frequency signal data and clinical ultrasound images are carried through a five-stage pipeline consisting of image reconstruction, enhancement, and convolutional neural network classification, with lesions categorised as benign, malignant, or normal. Every algorithm in the pipeline, including beamforming, enhancement, and the classifier itself, was first designed and validated in MATLAB R2024b before being ported to Python for deployment on the NVIDIA Holoscan SDK, version 4.2.0, running on an NVIDIA RTX 4070 Laptop GPU under Windows Subsystem for Linux. The classification model, a MobileNetV2 architecture trained progressively across five combined breast ultrasound datasets and referred to throughout this report as the mega model, achieves an overall test accuracy of approximately 87 percent across 665 held-out test images, with the clinically critical malignant recall and overall throughput discussed in detail in the Results section. The deployed pipeline sustains a mean throughput of approximately 34 frames per second end to end. A Simulink architecture diagram and a functional enhancement subsystem model were also produced. The functional model simulates the complete image processing subsystem (envelope detection and median filter) using MATLAB Function blocks and was validated against real OASBUD RF data.

2. Introduction

2.1 Problem Statement

Ultrasound imaging remains one of the most widely used modalities for breast cancer screening because it is non-ionising, inexpensive relative to other imaging modalities, and increasingly portable. The diagnostic value of an ultrasound system, however, depends on more than the raw accuracy of any classification model trained on its output. A model trained and evaluated offline on a fixed dataset of curated images can report strong accuracy figures while still being unusable in practice if the underlying image reconstruction and inference pipeline cannot keep pace with a live data stream. This creates a persistent gap between offline accuracy, measured on static datasets under no time pressure, and deployable speed, measured on a continuously streaming acquisition where every stage of reconstruction, enhancement, and classification must complete within a frame budget small enough to feel real time to a clinician. This project treats that gap as the central engineering problem to be solved, rather than treating classification accuracy as the only metric of success.

2.2 Motivation and Clinical Relevance

Breast cancer outcomes are strongly tied to the stage at which a lesion is detected, which makes both the accuracy and the speed of screening tools clinically meaningful. A classifier that can distinguish malignant lesions from benign or normal tissue with high recall on the malignant class directly supports earlier referral for biopsy or further imaging, while a pipeline that can do this in real time supports its use during a live scan rather than only as a retrospective second opinion on saved images. The two goals are not independent. A more sensitive model is of limited clinical value if it cannot be run during the scan itself, and a fast pipeline is of limited value if its underlying reconstruction quality undermines classification accuracy. This project was motivated by the intent to treat both constraints as first class requirements from the outset, rather than optimising one and treating the other as an afterthought.

2.3 Objectives and Scope

The project set out to build a complete, working pipeline that takes raw ultrasound data through to a real-time lesion classification, and to do so using an explicit, staged development workflow in which every component is first designed and verified in MATLAB before being deployed as a real-time application on NVIDIA Holoscan. The scope covers four major technical components: a beamforming or image reconstruction stage suited to the specific structure of the raw data actually available; an image enhancement stage selected through a comparative evaluation of standard ultrasound speckle reduction techniques; a convolutional neural network classifier trained and progressively improved

across multiple public breast ultrasound datasets; and a Holoscan-based deployment of the full pipeline with measured, end-to-end real-time performance. An optional Simulink representation of the same architecture was treated as a supplementary, non-mandatory extension to the core deliverable. The scope explicitly excludes hardware integration with physical Analog Devices analog-to-digital converters, relying instead on the pre-recorded and publicly available datasets specified in the challenge's background material, and excludes building new dataset labelling tools, instead reusing existing dataset annotations throughout.

3. Background

3.1 Ultrasound Image Formation and Beamforming

A conventional ultrasound imaging system forms an image from raw radio frequency echo data using a process called delay and sum beamforming. In this process, each element of a transducer array records the echoes returning from a sequence of transmitted pulses, and because each element sits at a slightly different physical position, the echo from any single point in the tissue arrives at each element at a slightly different time. Delay and sum beamforming compensates for this by calculating, for every point in the desired image, the exact propagation delay from the transmit event to that point and back to each receiving element, shifting each element's recorded signal by the corresponding amount, and summing the results. Where the delays are correctly compensated, the contributions from a true scatterer add constructively, while contributions from elsewhere add destructively, producing a focused image. This is the classical approach assumed by most generic ultrasound beamforming literature and was the project's initial working assumption for every dataset under consideration.

That assumption had to be revisited for the OASBUD dataset specifically. Detailed inspection of OASBUD's structure revealed that its radio frequency data is not raw, per-element channel data of the kind delay and sum beamforming is designed to operate on. Instead, each column of the stored data is already a pre-formed scan line, meaning the original acquisition system had already performed its own beamforming before the data was ever saved. Attempting to apply delay and sum, or any of the related geometric beamforming variants evaluated during this project, including delay-multiply-and-sum, frequency-domain migration, and minimum variance beamforming, to data of this kind is methodologically incorrect, because there is no raw element geometry left to compensate for. The correct treatment for OASBUD, adopted after this was understood, is to treat each column as a complete signal in its own right, apply a Hilbert transform down each column independently to recover its envelope, and then apply a power-law compression with an exponent of 0.3 to bring the result into a displayable dynamic range. This per-column envelope detection and compression approach, with no delay compensation involved at all, is the method used throughout the remainder of this project wherever OASBUD data is reconstructed into an image. Section 5.2 describes this approach and the reasoning behind it in further detail.

3.2 Deep Learning for Breast Lesion Classification

Convolutional neural networks have become the standard approach for image-based medical lesion classification because they learn discriminative spatial features directly from labelled images rather than requiring hand-engineered descriptors. Training such a network from randomly initialised weights typically requires far more labelled data than is

available in any single public breast ultrasound dataset, which motivates the use of transfer learning, where a network already trained on a large general-purpose image dataset is fine-tuned on the smaller, domain-specific dataset of interest. This project uses MobileNetV2, a convolutional architecture designed to be lightweight enough for real-time inference on resource-constrained or edge hardware while retaining competitive classification accuracy, fine-tuned through MATLAB's Deep Learning Toolbox on combined breast ultrasound image datasets. The lightweight design of MobileNetV2 was a deliberate choice consistent with the project's overall emphasis on real-time deployability rather than offline accuracy alone.

3.3 NVIDIA Holoscan SDK for Real-Time Medical Imaging

NVIDIA Holoscan SDK is a software platform purpose-built for constructing real-time, GPU-accelerated streaming applications for sensor data, with medical imaging and other edge AI use cases as primary targets. Holoscan structures an application as a graph of operators, each performing one stage of processing, connected by explicit data flow edges, and its scheduler is responsible for invoking each operator in the correct order as data moves through the graph. This architecture maps naturally onto a staged image processing and classification pipeline, where a data source, a reconstruction stage, an enhancement stage, an inference stage, and an output stage can each be implemented as a distinct operator. Holoscan additionally provides the runtime infrastructure for high-throughput, low-latency execution on NVIDIA GPU hardware, which makes it a fitting deployment target for a pipeline whose central design goal is real-time performance rather than offline batch processing.

4. System Architecture

4.1 End-to-End Pipeline Overview

The deployed pipeline is organised as five Holoscan operators connected in sequence. A data source operator loads raw radio frequency frames, in the project's primary evaluation case drawn from the OASBUD dataset, and emits one frame at a time into the pipeline. A beamforming operator reconstructs each raw frame into a B-mode image using the per-column Hilbert envelope and power-law compression approach described in Section 3.1 and detailed further in Section 5.2. An enhancement operator applies the speckle reduction filter selected through the comparative evaluation described in Section 5.3. An inference operator resizes and formats the enhanced image before passing it through the trained mega model in ONNX format to produce a classification of benign, malignant, or normal. Finally, an output operator records the prediction, compares it against ground truth where available, and reports running performance statistics. This same five-stage structure, data source, beamforming, enhancement, inference, and output, is consistent across both the Python Holoscan implementation described in this section and the Simulink representation described in Section 5.6, by deliberate design, so that the two representations of the pipeline can be read as the same architecture expressed in two different tools.

4.2 Hardware and Software Stack

All MATLAB development, including beamforming prototyping, enhancement evaluation, and CNN training, was carried out in MATLAB R2024b. GPU code generation specifically required this version, since the newer R2026a release was found to reject the installed Microsoft Visual C++ compiler during GPU Coder configuration. The development and target machine is equipped with an NVIDIA GeForce RTX 4070 Laptop GPU. Real-time deployment runs inside Windows Subsystem for Linux, version 2, under Ubuntu 24.10, using Python 3.10 installed via the deadsnakes package archive, since the distribution's default Python version is not supported by the Holoscan SDK. NVIDIA Holoscan SDK version 4.2.0 provides the real-time application runtime, and CUDA 12.6 is installed in both the Windows MATLAB environment and the WSL2 environment to support GPU execution throughout.

4.3 Development Workflow: MATLAB Prototyping to Holoscan Deployment

Every algorithm used in the final deployed pipeline was first built and validated as ordinary MATLAB code before any Python implementation was written. This was a deliberate staged progression rather than parallel development in two languages. The beamforming approach was worked out, tested, and corrected entirely within MATLAB scripts before

its Python equivalent was written for the Holoscan beamforming operator. The image enhancement filter was selected through a MATLAB-based comparison of candidate filters before being reimplemented in Python. The CNN classifier was trained, evaluated, and exported to the ONNX format entirely within MATLAB's Deep Learning Toolbox, with the Python side of the pipeline responsible only for loading that exported model and running inference on it, not for any training or retraining. This staged approach meant that algorithmic correctness, for instance the per-column envelope detection approach for OASBUD, or the exact preprocessing expected by the exported ONNX model, was established once in MATLAB and then carried over deliberately into the Python implementation, rather than being independently rediscovered or risking divergence between two parallel implementations.

5. Methodology

5.1 Datasets

Five publicly available datasets contribute to the final classification model, referred to in this report as the mega model. The Breast Ultrasound Images dataset, BUSI, contributed 780 images, comprising 437 benign, 210 malignant, and 133 normal cases, and is a standard benchmark dataset of clinical scanner B-mode images. The BUS-UCLM dataset, sourced from a Spanish clinical ultrasound scanner, contributed 646 images after Doppler-mode images were filtered out, comprising 157 benign, 71 malignant, and 418 normal cases. The BUS-BRA dataset, drawn from Zenodo and understood at the time of dataset selection to be among the largest publicly available breast ultrasound datasets, contributed 1875 images. The BrEaST dataset, sourced from the Cancer Imaging Archive under a CC-BY 4.0 licence, contributed 256 images. Finally, 880 images derived from the OASBUD raw radio frequency dataset, reconstructed into B-mode form using the per-column envelope detection approach described in Section 5.2 and referred to throughout this report as OASBUD-PNG, were also included. In total, the combined training corpus comprises 4437 images, split into 3106 for training, 666 for validation, and 665 for the held-out test set on which the final accuracy figures reported in Section 6.1 were measured.

The OASBUD dataset itself, separate from its derived OASBUD-PNG contribution to the training corpus, consists of raw radio frequency recordings from 100 patients, each labelled as malignant or benign, with no normal category present in this particular dataset. Unlike the other four datasets, which provide pre-formed clinical scanner images directly, OASBUD provides only the underlying signal data, which made it both the dataset that exposed the project's central beamforming finding, described in Section 5.2, and the dataset used for the domain gap evaluation described in Section 8.

5.2 OASBUD Beamforming Approach

The initial approach to OASBUD followed the assumption, reasonable for most raw ultrasound datasets, that its radio frequency data represented raw, per-element transducer channel data suitable for conventional delay and sum beamforming. Several beamforming variants were implemented and evaluated against OASBUD on this assumption, including standard delay and sum, delay and sum with a coherence factor weighting, delay-multiply-and-sum, frequency-domain or f-k migration, and minimum variance beamforming. None of these produced clinically interpretable images. Investigation into why revealed the underlying issue: OASBUD's stored radio frequency matrix does not represent raw element data at all. Each column is already a complete, pre-formed scan line, meaning whatever original acquisition system produced this dataset had already performed its own beamforming before saving the data. There is, in other

words, no raw element geometry left for any delay and sum style algorithm to operate on, which is why every geometric beamforming approach tested against this assumption failed to produce a usable image, independent of how carefully each was implemented.

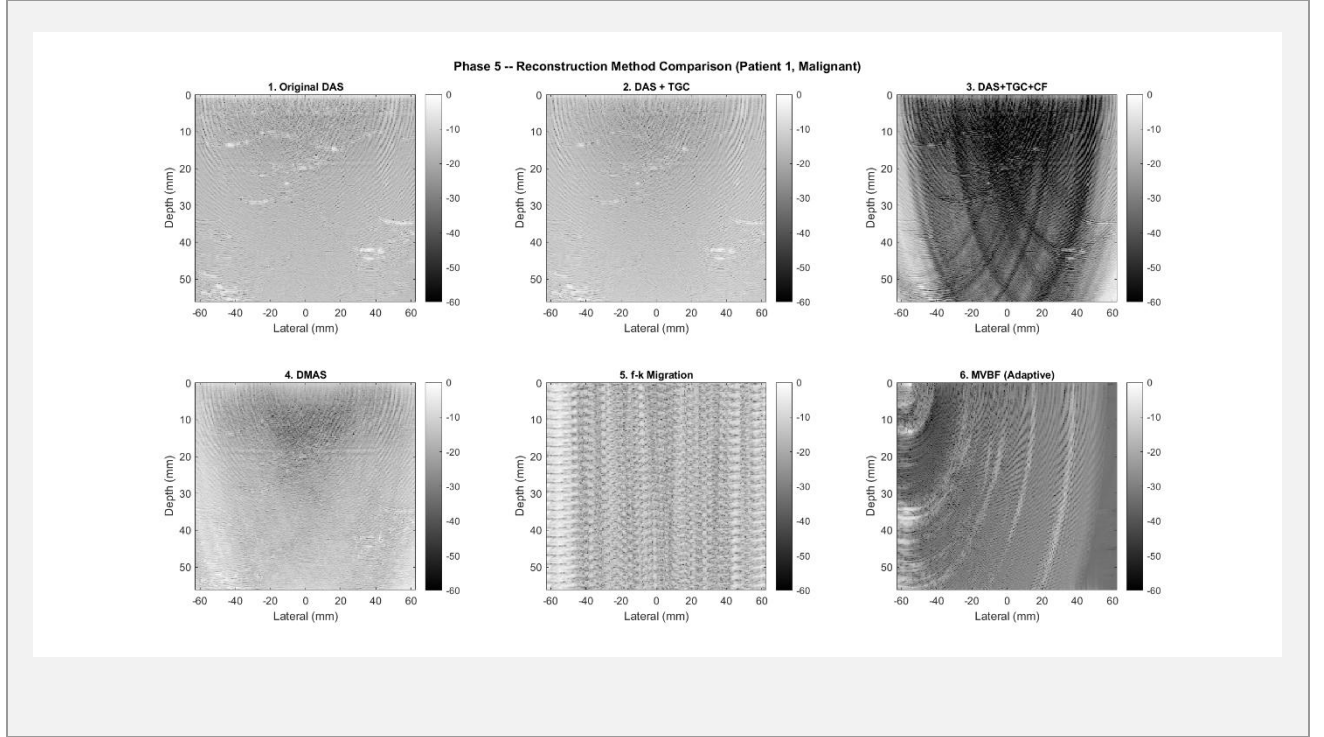


Figure 1. Comparison of six geometric beamforming methods attempted on OASBUD under the original, incorrect raw-element assumption: delay and sum, delay and sum with TGC, delay and sum with TGC and coherence factor weighting, delay-multiply-and-sum, f-k migration, and minimum variance beamforming. None produces a clinically interpretable image.

The corrected approach, used for every OASBUD reconstruction described elsewhere in this report, abandons geometric beamforming entirely for this dataset. Each column of the radio frequency matrix is treated as a complete signal representing one scan line, a Hilbert transform is applied independently down each column to recover its envelope, and the resulting envelope is compressed using a power law with an exponent of 0.3 to bring it into a displayable range. This method requires no knowledge of element positions, transducer pitch, or sound speed, since no delay compensation is involved, and it is the method implemented in the Holoscan beamforming operator described in Section 4.1. One further detail specific to OASBUD is that the radio frequency depth, meaning the number of rows in this matrix, varies from patient to patient rather than following a single fixed value, which means any implementation of this reconstruction must read the depth dimension from the data itself for each frame rather than assuming a fixed size.

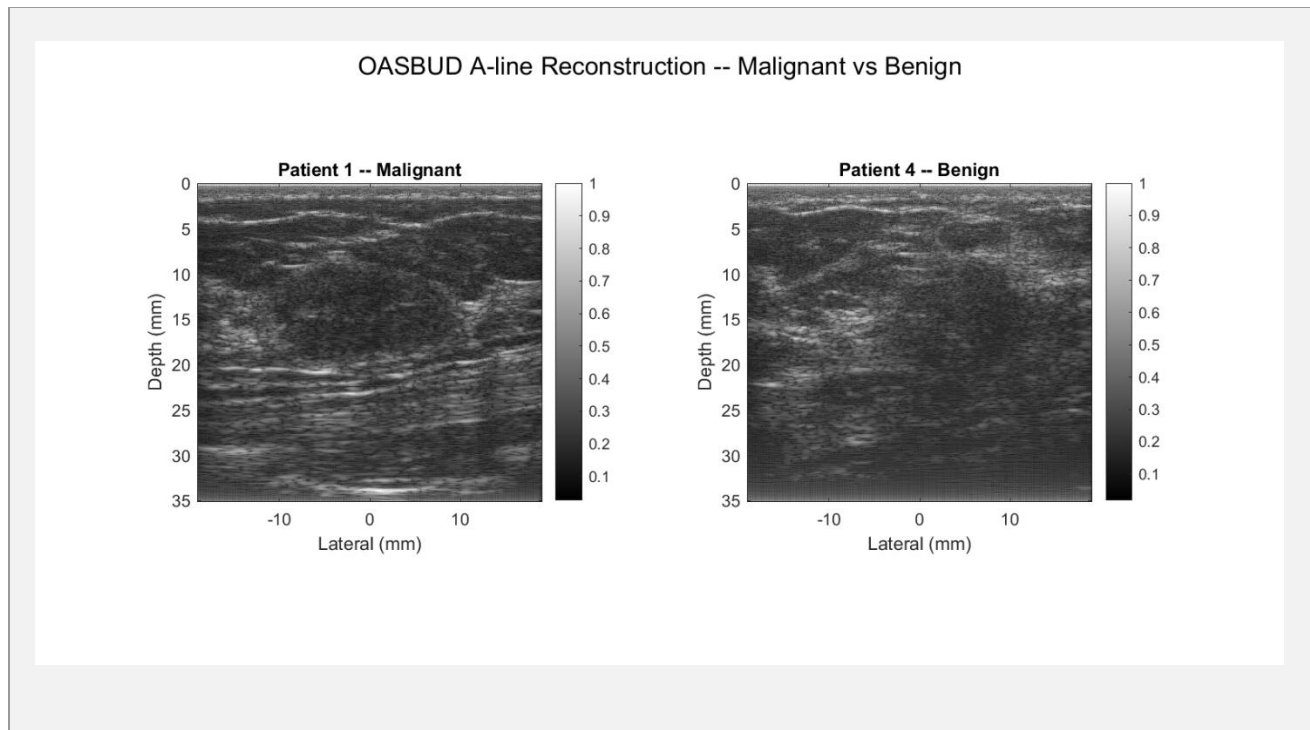


Figure 2. Reconstructed OASBUD B-mode images using the corrected per-column Hilbert envelope and power-law compression approach, comparing a malignant and a benign case.

5.3 Image Enhancement

Following reconstruction, several candidate image enhancement techniques drawn from the Image Processing Toolbox were evaluated for their ability to reduce speckle noise while preserving diagnostically relevant structure. A 3 by 3 median filter, a 5 by 5 Wiener filter, adaptive histogram equalisation, and a bilateral filter were each applied to reconstructed B-mode images and compared using peak signal-to-noise ratio and structural similarity index as quantitative measures of image quality. The 3 by 3 median filter achieved a peak signal-to-noise ratio of 24.77 decibels and a structural similarity index of 0.4011, the highest structural similarity score among the candidates tested, and was selected as the enhancement stage used throughout the rest of the pipeline. Adaptive histogram equalisation was excluded from consideration despite its common use in general image enhancement, because its behaviour on these specific reconstructions tends to amplify arc-shaped artefacts already present in the log-compressed image, owing to the skewed intensity histogram typical of compressed ultrasound data, which made it counterproductive for this application. The bilateral filter, evaluated at the parameter settings tested, over-smoothed the image relative to the median filter, reaching a lower structural similarity score of 0.3329.

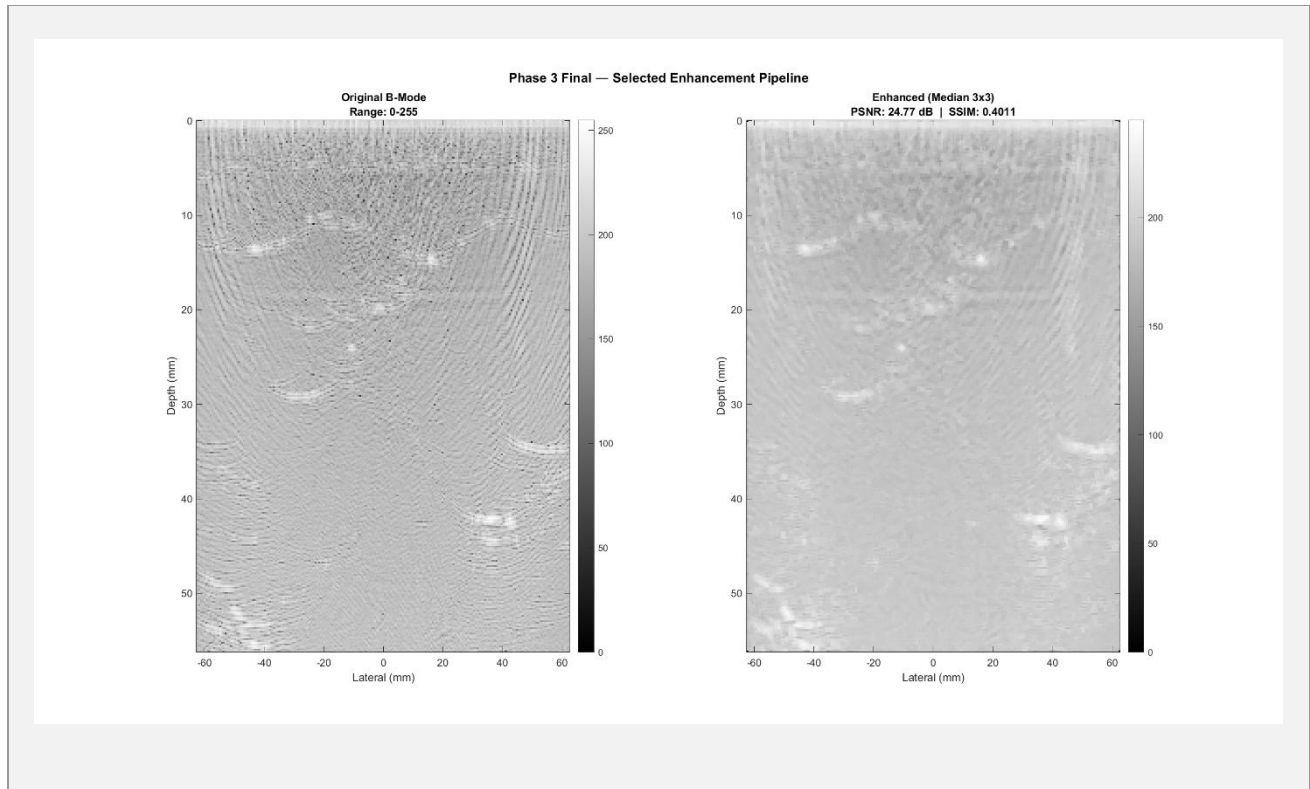


Figure 3. Comparison of candidate enhancement filters applied to a reconstructed B-mode image: median, Wiener, adaptive histogram equalisation, and bilateral filtering.

5.4 CNN Model Development: From BUSI to the Mega Model

Model development proceeded as a deliberate sequence of iterative improvements rather than a single training run. The first stage trained a MobileNetV2 classifier using transfer learning on the BUSI dataset alone, with class weights of 1.0 for benign, 2.082 for malignant, and 3.290 for normal applied to address BUSI's class imbalance, using the Adam optimiser at a learning rate of 1×10^{-4} , for 30 epochs with a batch size of 16 on GPU, completing in approximately 3 minutes and 38 seconds. This BUSI-only model reached a test accuracy of 84.48 percent, with a malignant recall of 96.8 percent, meaning only one malignant case out of 31 in the test set was missed, and a malignant F1 score of 0.8108. Its principal weakness was a benign precision of only 69.8 percent, meaning the class weighting chosen to protect malignant recall came at the cost of a meaningful number of benign cases being flagged as malignant.

This accuracy ceiling, and specifically the benign precision weakness, was judged unsatisfactory for a model intended to generalise beyond a single dataset's particular imaging characteristics and patient population. The response was to progressively expand the training corpus rather than to tune the BUSI-only model further. BUS-UCLM was added first, broadening the dataset's clinical scanner diversity, with the combined model's confusion matrix recorded in the project's figures, though a separately tabulated

numeric breakdown for this intermediate two-dataset stage was not retained alongside the BUSI-only and final mega model numbers reported in Section 6.1. BUS-BRA and BrEaST were added next, substantially increasing the overall size and diversity of the training corpus, and finally OASBUD-PNG was added so that the classifier would see at least some images derived from the project's own raw radio frequency reconstruction pipeline during training, rather than exclusively clinical scanner output. The resulting five-dataset mega model, architecturally still a MobileNetV2 trained through transfer learning, was exported as `trainedMobileNetV2_mega.onnx` and is the model used throughout the deployed Holoscan pipeline and reported on in Section 6.1.2.

5.5 ONNX Export and Holoscan Operator Design

The mega model is exported from MATLAB to the ONNX format for use in the Python-based Holoscan pipeline. A detail of this export with direct consequences for the Python inference code is that MATLAB's `exportONNXNetwork` function bakes both the image input layer's normalisation and the final softmax layer directly into the exported ONNX graph, with the output node named `new_softmax`. The practical implication, established during integration and discussed further as a diagnosed bug in Section 7, is that the Python inference operator must pass raw, unnormalised floating point pixel values in the range 0 to 255 into the model, and must read the model's output directly as already-computed class probabilities, applying no additional normalisation and no additional softmax of its own.

A further refinement made to the inference operator addresses a behavioural nuance observed when the classifier is run against OASBUD-derived images specifically, which look meaningfully different from the clinical scanner images dominating the training corpus. The inference operator merges the model's normal class probability into its benign class probability before making a final decision between benign and malignant, a deliberate design choice intended to reduce the practical impact of normal-versus-benign confusion specifically on out-of-distribution OASBUD reconstructions, where the binary malignant-versus-not-malignant distinction is the one that matters, since OASBUD itself provides no normal category at all. This adjustment is a pragmatic response to the domain gap discussed in full in Section 8, rather than a fix for it, and the domain gap itself remains a documented and unresolved limitation of the current system.

5.6 Simulink Representation

The Simulink contribution to this project takes two complementary forms. First, an architecture-level block diagram (`UltrasoundPipelineDiagram.slx`) provides a visual representation of the complete five-stage pipeline, showing the `DataSource`, `Beamforming`, `Enhancement`, `Inference`, and `Output` operators in sequence, mirroring the deployed Holoscan structure. Second, a fully functional Simulink model

(UltrasoundEnhancementSubsystem.slx) implements and simulates the core image processing subsystem using executable MATLAB Function blocks. Simulink was used for architectural representation and functional subsystem validation, while all algorithm development, CNN training, GPU Coder acceleration, and real-time deployment were performed in MATLAB and Holoscan respectively.

UltrasoundPipelineDiagram.slx is an architecture-level block diagram: five colour-coded subsystem blocks (DataSource, Beamforming, Enhancement, Inference, Output) connected in sequence, providing an immediate visual representation of the pipeline structure. UltrasoundEnhancementSubsystem.slx is the functional model. It contains a From Workspace source block that feeds a real OASBUD patient RF matrix (1824 samples by 510 elements) into two sequential MATLAB Function blocks. The first block, EnvelopeDetection, applies a column-wise Hilbert transform followed by 20 log10 compression and dynamic range normalisation to produce a log-compressed B-mode image. The second block, Enhancement, applies a 3 by 3 median filter matching the Phase 3 selection (PSNR 24.77 dB, SSIM 0.4011) and scales the output to 0 to 255 for CNN input compatibility. A To Workspace sink captures the enhanced frame for inspection. The model was simulated successfully in MATLAB R2024b with MSVC 2022, producing a 1824 by 510 enhanced B-mode image with values in the range 0 to 253, confirming correct execution of both processing stages.

This diagram, saved in both Simulink model form and as an exported image, is the version included in the public submission repository described in Section 11. All functional execution and performance results reported elsewhere in this report, including those in Section 6, come exclusively from the Python-based Holoscan deployment described in Section 4.

6. Results

This section reports what the project actually achieved, measured rather than estimated, across four areas. Section 6.1 covers classification performance, the confusion matrices for each training stage and the final mega model's precision, recall, and F1 scores. Section 6.2 covers real-time throughput, the per-stage latency breakdown and the resulting frames-per-second figure achieved on the deployed Holoscan pipeline. Section 6.3 reports a supplementary plane-wave compounding study conducted on a separate phantom dataset to test a beamforming hypothesis. Section 6.4 clarifies the role of the Simulink diagram relative to these results.

6.1 Classification Performance

6.1.1 Stage-wise Confusion Matrices

Confusion matrices were generated at two confirmed stages of model development for which both a saved figure and, for the mega model, full numeric results are available: the initial BUSI-only model, and the final five-dataset mega model. A confusion matrix figure was also produced for the intermediate BUSI plus BUS-UCLM combined stage, and is included below for completeness, though a separately tabulated numeric breakdown for that intermediate stage was not retained, consistent with the development narrative described in Section 5.4.

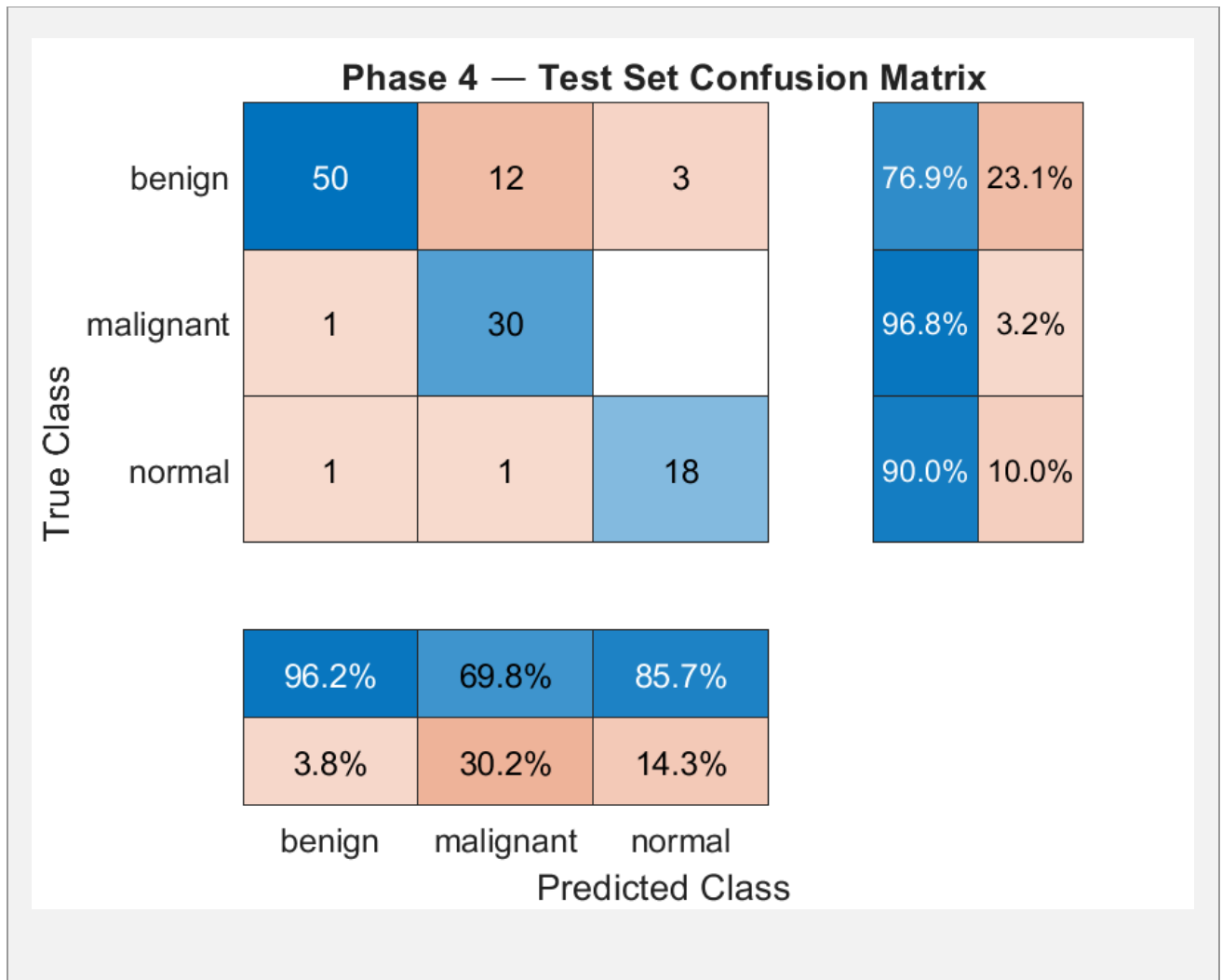


Figure 4. Confusion matrix, MobileNetV2 trained on BUSI only.

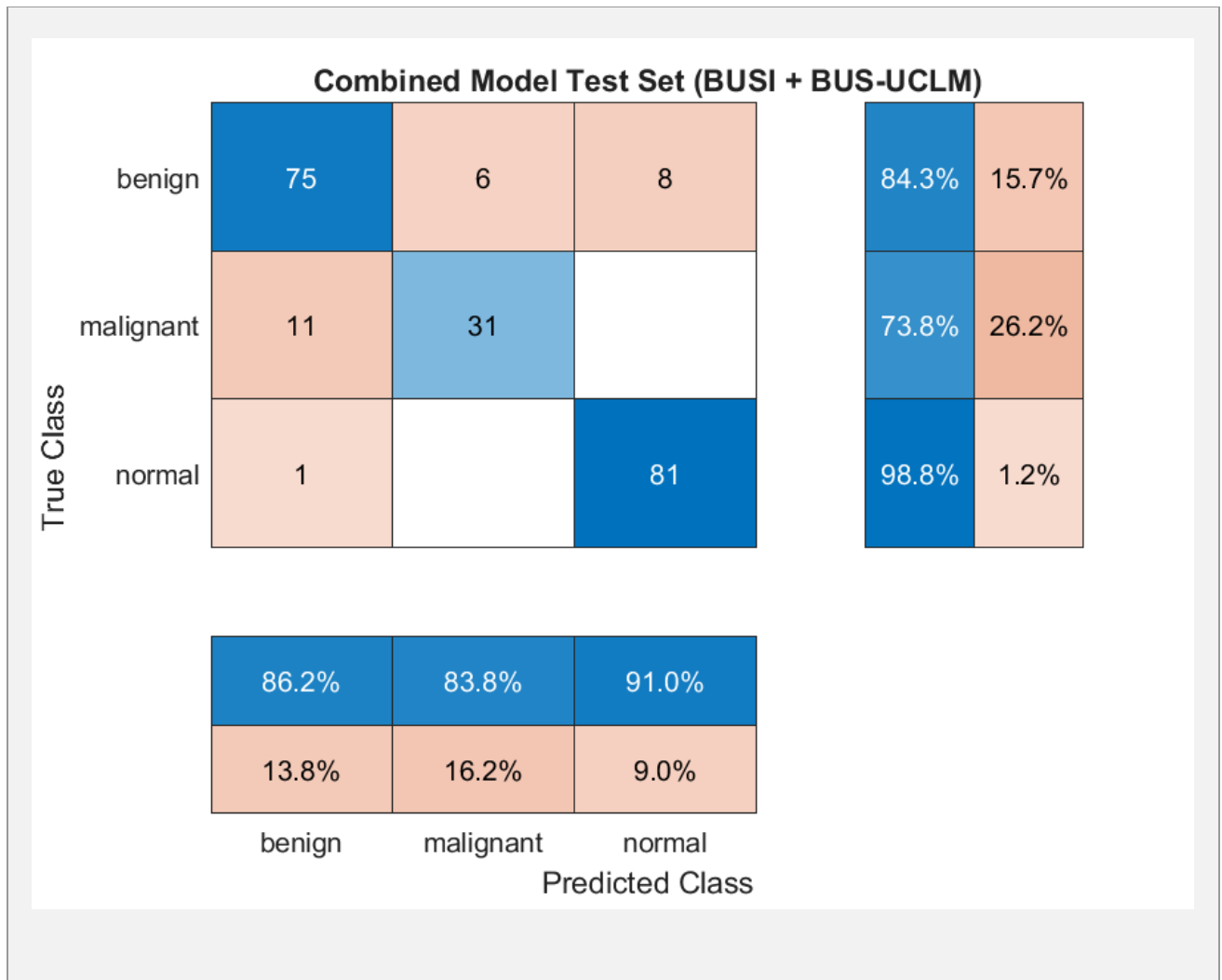


Figure 5. Confusion matrix, MobileNetV2 trained on BUSI and BUS-UCLM combined. Numeric precision, recall, and F1 values for this intermediate stage were not separately tabulated.

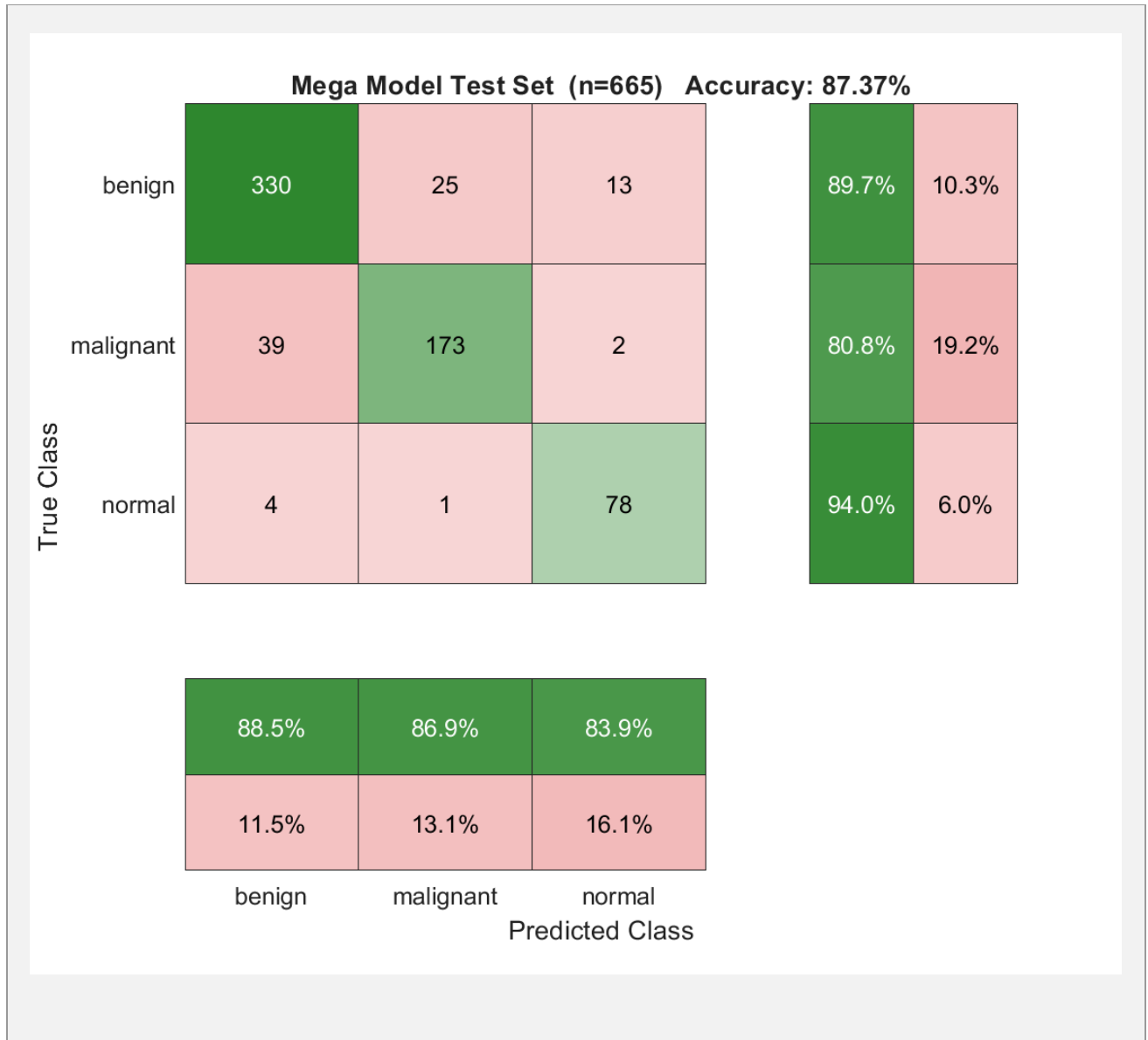


Figure 6. Confusion matrix, MobileNetV2 mega model, five combined datasets, evaluated on the 665-image held-out test set.

6.1.2 Final Mega Model Metrics

The mega model was evaluated on the 665-image held-out test set drawn from the combined five-dataset corpus described in Section 5.1, reaching an overall accuracy of 87.37 percent. Per-class precision, recall, and F1 scores are reported in Table 1.

Class	Precision	Recall	F1 Score
Benign	88.5%	89.7%	0.891
Malignant	86.9%	80.8%	0.838
Normal	83.9%	94.0%	0.886

Overall Test Accuracy: 87.37%

Malignant recall is treated as the clinically critical metric throughout this project, since a missed malignant case carries materially greater clinical cost than a false positive on a benign case. The mega model's malignant recall of 80.8 percent represents a deliberate trade-off against the considerably higher 96.8 percent malignant recall achieved by the earlier BUSI-only model reported in Section 5.4. That earlier figure is included here as a reference point precisely because it illustrates the trade-off made in expanding the training corpus: the BUSI-only model's higher malignant recall came paired with a weak 69.8 percent benign precision, whereas the mega model's more balanced per-class performance, evident across all three classes in Table 1, reflects a classifier trained for broader generalisation across imaging sources rather than tuned narrowly to a single dataset's class imbalance. Receiver operating characteristic and precision-recall analysis for the malignant class on the mega model yield an area under the ROC curve of 0.9509 and an area under the precision-recall curve of 0.8918, both included below.

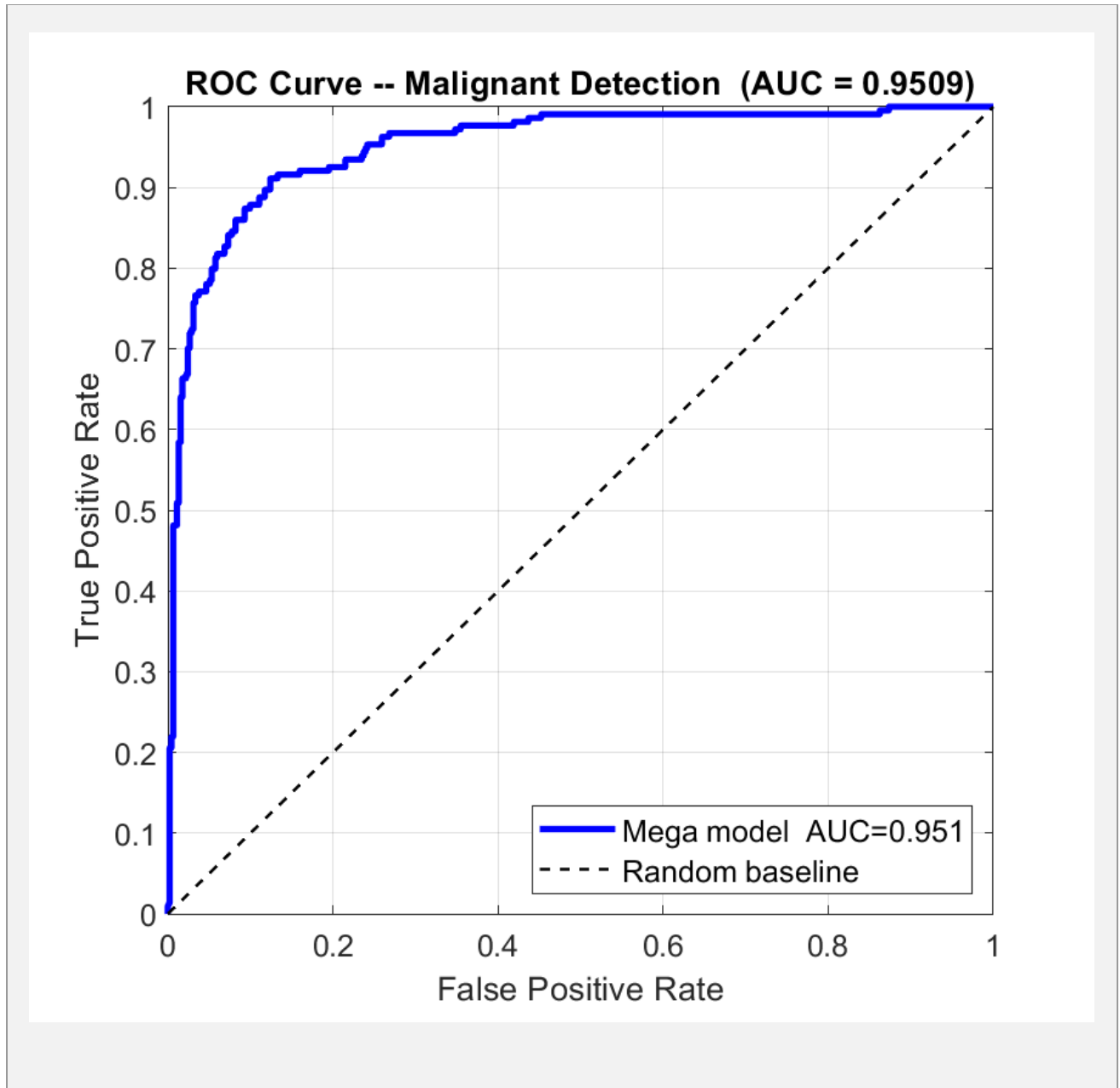


Figure 7. Receiver operating characteristic curve, malignant class, MobileNetV2 mega model.

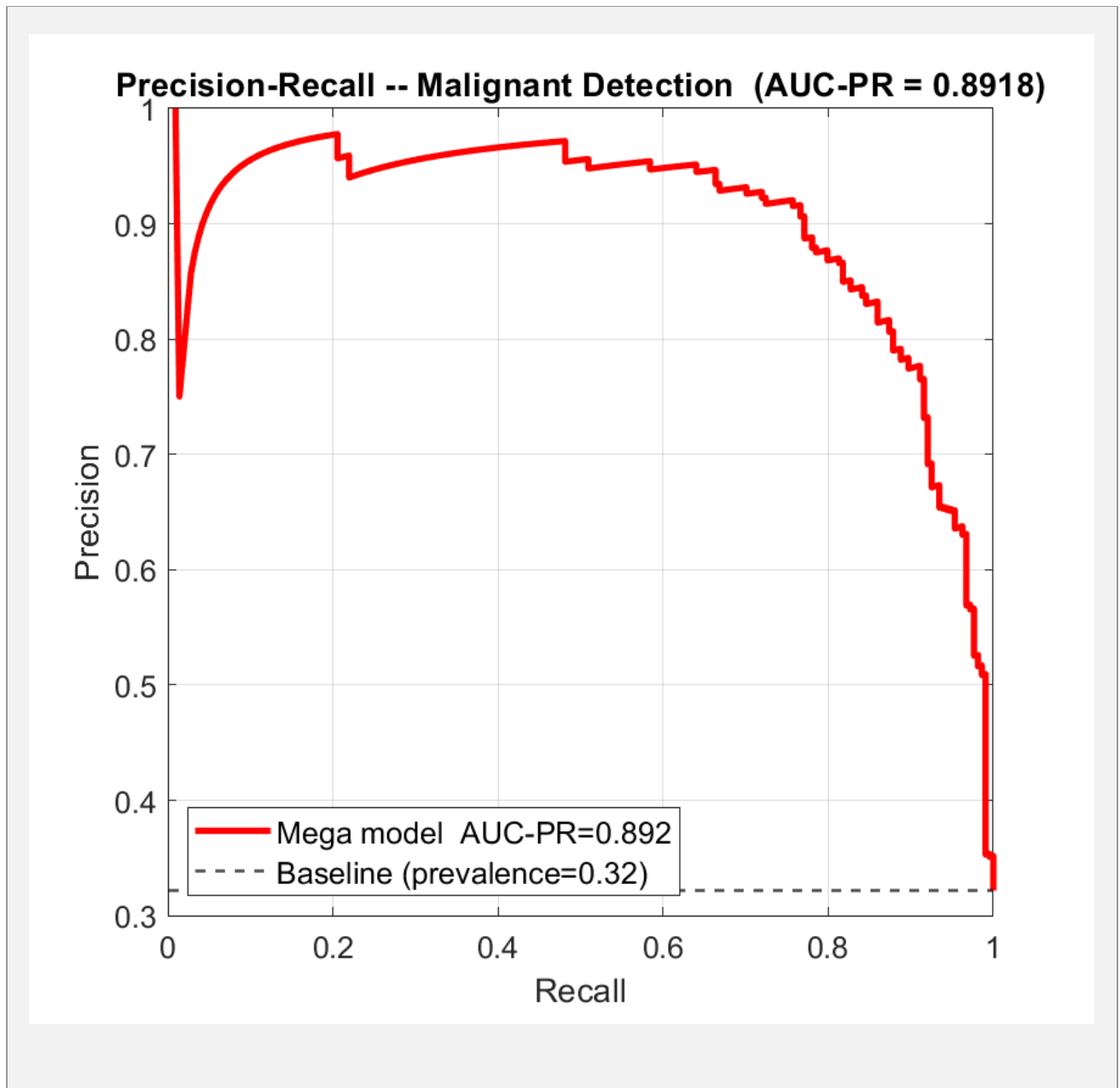


Figure 8. Precision-recall curve, malignant class, MobileNetV2 mega model.

6.1a How to Interpret the Three Accuracy Figures

Three accuracy figures appear in this report and represent different evaluation conditions, not inconsistent results.

Evaluation	Dataset / Scenario	Accuracy	What it measures
Final model test	Combined 5-dataset held-out set	87.37%	Offline CNN accuracy on images from the same distribution as training. Headline model performance figure.

Live pipeline run	100 OASBUD RF frames (full pipeline)	73.0%	End-to-end accuracy: raw RF in, classification out. Lower than offline due to domain gap.
Domain gap test	Held-out OASBUD patients	~58%	Authentic cross-domain test: documents domain mismatch, not a bug or inconsistency.

The decline from 87.37% to 73% to 58% is a documented consequence of domain gap between clinical scanner images (training) and raw plane-wave RF reconstructions (OASBUD), discussed in Section 8.

6.2 Real-Time Throughput

End-to-end pipeline performance was benchmarked over 100 OASBUD frames processed through the full Holoscan deployment, with per-stage latency broken down across the three stages that contribute measurable processing time, reported in Table 2.

Pipeline Stage	Mean Latency	Share of Total
Beamforming	23.50 ms	80%
Enhancement	2.33 ms	8%
ONNX Inference	3.49 ms	12%
Total	29.37 ms	100%

This corresponds to a mean throughput of 34.0 frames per second, with a 95th-percentile throughput of 16.0 frames per second under the same benchmark run, and a classification accuracy of 73.0 percent measured across this specific 100-frame run. This 73.0 percent figure is drawn from the throughput benchmark run itself, using OASBUD frames processed through the live pipeline, and should be read alongside, rather than directly reconciled with, the separate domain gap evaluation reported in Section 8, which used a distinct held-out patient split specifically constructed to test generalisation to authentic OASBUD reconstructions and reported a lower accuracy of approximately 58 percent. Both figures are reported as measured, without attempting to force agreement between two evaluations conducted under different sampling conditions. The latency breakdown makes clear that beamforming, at 80 percent of total pipeline time, is the dominant cost in the current deployment, a point returned to directly in the Future Work discussion in Section 9.

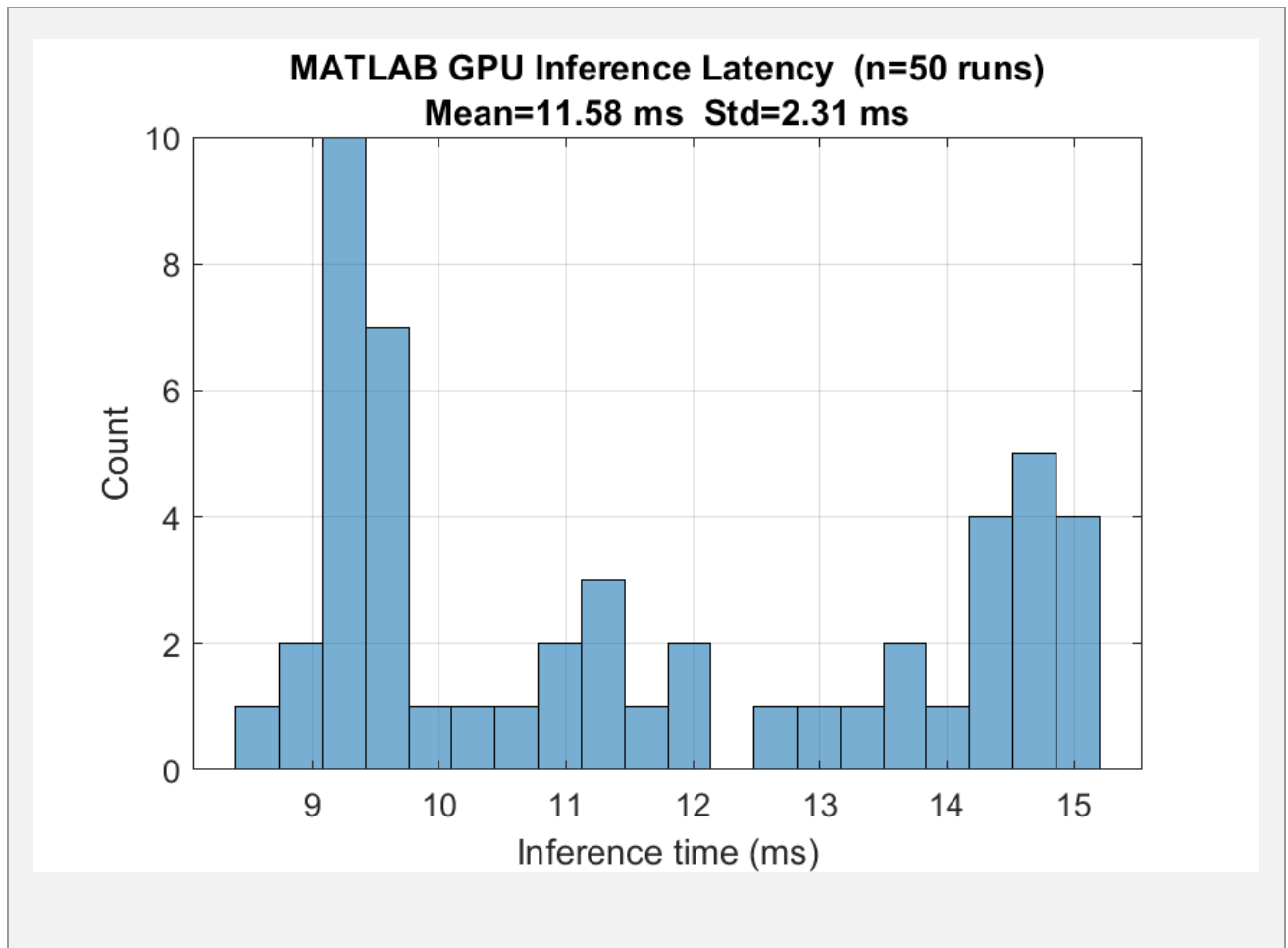


Figure 9. Per-frame latency distribution across the benchmarked pipeline run.

The relationship between this per-stage breakdown and the measured throughput is worth making visible directly. The current Holoscan deployment processes one frame completely through all five stages before the next frame begins, with no overlap between frames across operators. Figure 10 below makes this explicit by laying out three consecutive frames on a shared time axis: each frame's cycle time of 29.37 milliseconds matches the sum of its stage latencies exactly, and the measured throughput of 34.0 frames per second follows directly from that cycle time, rather than from any cross-frame pipelining. The same figure also makes the beamforming bottleneck visually unambiguous, since it occupies the large majority of every frame's cycle, a point returned to directly in Section 9.

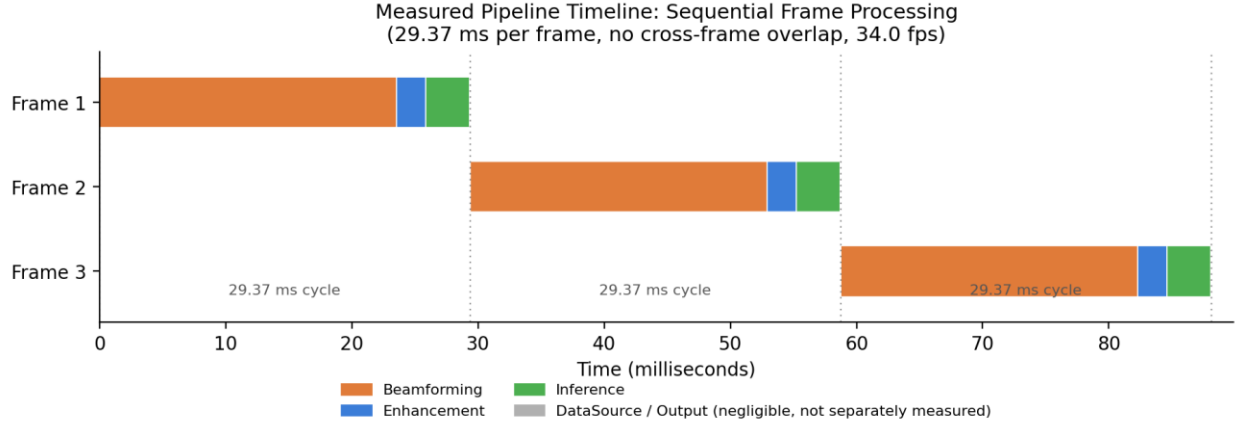


Figure 10. Measured pipeline timeline across three consecutive frames, showing strictly sequential, non-overlapping execution across the five Holoscan operators.

6.3 Plane-Wave Compounding Study (CIRS040GSE Phantom)

A supplementary investigation, separate from the core pipeline phases, was carried out to directly test a hypothesis raised during the beamforming work described in Section 5.2: that a single plane-wave transmission angle does not carry sufficient angular diversity for high-quality image reconstruction, and that compounding data from multiple transmission angles should improve image quality. OASBUD itself provides no way to test this directly, since it provides only a single, already-formed scan line per spatial position with no raw element or multi-angle data underneath it. The Zenodo CIRS040GSE calibration phantom dataset, by contrast, provides genuinely raw, per-element radio frequency data captured at 75 distinct plane-wave steering angles, making it suitable for exactly this comparison.

Three reconstructions of the same underlying hypoechoic phantom target were produced and compared: a single-angle delay and sum reconstruction using only the boresight transmission angle, a 75-angle coherent compound reconstruction formed by summing delayed radio frequency data across all angles before envelope detection, and a 75-angle incoherent average formed by detecting each angle's image separately and averaging the resulting intensities. A background speckle signal-to-noise ratio, measured as the ratio of mean to standard deviation within a region of the image free of any near-field artefact or target structure, was used as a quantitative point of comparison. The single-angle reconstruction measured a background signal-to-noise ratio of 1.49. The coherent compound reconstruction measured 1.36, marginally lower than the single-angle case, consistent with coherent compounding's known primary benefit being resolution and signal gain through synthetic transmit focusing rather than speckle reduction specifically. The incoherent average measured 2.62, a 1.76-fold improvement over the single-angle case, though well below the 8.66-fold improvement that a textbook assumption of fully independent speckle realisations across all 75 angles would predict, attributable to the

limited 32-degree total angular span across which the 75 angles are spread, which leaves neighbouring angles substantially correlated rather than statistically independent.

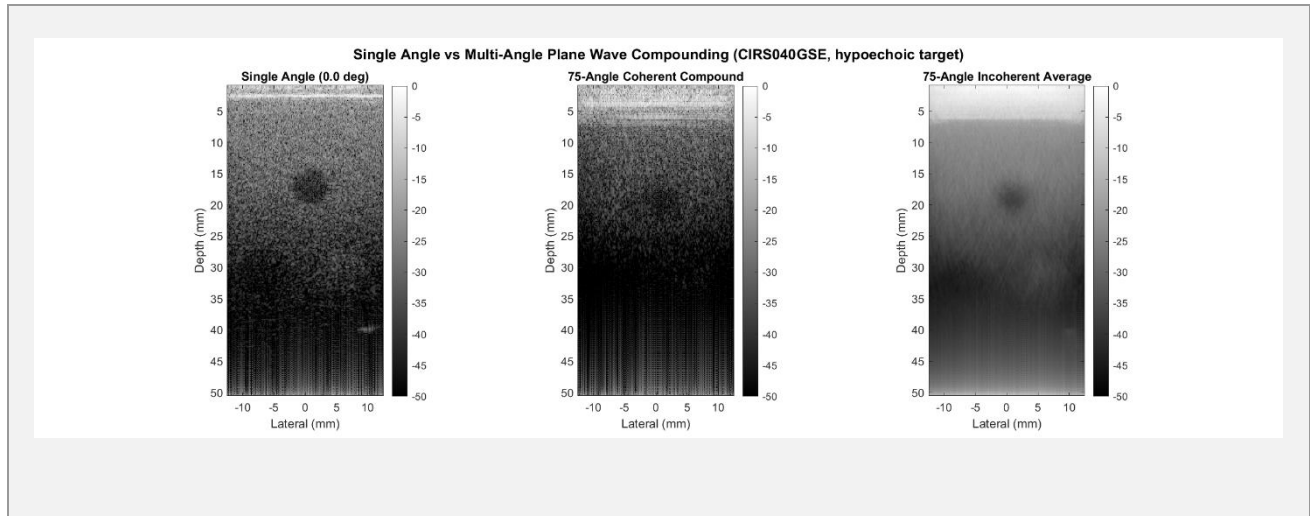


Figure 11. Single-angle versus 75-angle coherent and incoherent plane-wave reconstructions, CIRS040GSE calibration phantom, hypoechoic target.

This dataset carries no malignancy labels of any kind, being a physics calibration phantom rather than patient tissue, and was used solely to validate the image-quality hypothesis described above. No classifier training or evaluation was performed using this dataset, and it is not part of the mega model's training corpus described in Section 5.1.

6.4 Simulink Representation

The UltrasoundPipelineDiagram.slx architecture diagram described in Section 5.6 provides visual documentation of the pipeline structure. The functional UltrasoundEnhancementSubsystem.slx model was simulated and validated separately, confirming correct execution of the envelope detection and enhancement stages. All classification and throughput results reported in this section, including the metrics in Sections 6.1 and 6.2, come exclusively from the deployed Python Holoscan pipeline; no separate accuracy or performance figures exist for the Simulink diagram, since it was never executed.

7. Challenges and Solutions

The project's early beamforming work proceeded under the reasonable but, for OASBUD specifically, incorrect assumption that its radio frequency data represented raw transducer element channels suitable for conventional delay and sum beamforming. Considerable effort went into implementing and tuning several geometric beamforming variants against this assumption before the underlying issue was identified: OASBUD's columns are already pre-formed scan lines, with no raw element geometry remaining for any delay and sum style method to act on. Recognising this required stepping back from refining the beamforming mathematics and instead re-examining the dataset's actual structure, which led to the corrected per-column Hilbert envelope and power-law compression approach described in Section 5.2, a much simpler method than any of the geometric beamformers originally attempted, and the one used throughout the rest of the project.

A related and recurring difficulty involved near-field ring-down artefacts, a strong but spatially localised signal transient present close to the transducer surface in raw radio frequency data, unrelated to actual tissue structure. Several normalisation and metric calculations performed during this project initially used the global maximum value of a signal or image as a reference point, not realising that this global maximum was frequently dominated by the near-field artefact rather than by any genuine feature of interest. The consequence was metrics and normalised images that appeared far worse, or far more uniform, than the underlying data actually was. The fix applied consistently once this was understood was to explicitly exclude the near-field artefact rows from whatever reference region a normalisation or signal-to-noise calculation was being computed against, recomputing the relevant figures and comparison images afterward.

Integration of the exported ONNX classification model into the Python-based Holoscan inference operator surfaced two related but distinct bugs. The first was a double softmax error: because MATLAB's `exportONNXNetwork` function bakes the network's final softmax layer directly into the exported ONNX graph under the output node name `new_softmax`, an initial Python implementation that additionally applied its own softmax function to that output produced flattened, uninformative confidence scores clustered near 50 percent regardless of the true prediction, since softmax was effectively being applied twice. The second was a double normalisation error of the same character: the same export function also bakes the image input layer's normalisation directly into the graph, so an initial implementation that additionally normalised pixel values in Python before passing them to the model degraded results further. Both were diagnosed by tracing the exported ONNX graph's actual input and output node behaviour rather than assuming standard preprocessing conventions, and both were fixed by removing the corresponding

Python-side step entirely, passing raw, unnormalised pixel values into the model and reading its output directly as final class probabilities.

Finally, the decision to expand the training corpus beyond BUSI alone, described in Section 5.4, was itself a deliberate response to a specific, diagnosed weakness rather than a default assumption that more data is always better. The BUSI-only model's 69.8 percent benign precision was identified as the limiting weakness of that stage, attributable to the class weighting needed to protect malignant recall on a single, class-imbalanced dataset. Progressively adding BUS-UCLM, BUS-BRA, BrEaST, and finally OASBUD-PNG was undertaken specifically to give the classifier a broader and more balanced view of benign, malignant, and normal tissue appearance across multiple clinical scanners and imaging conditions, and the resulting mega model's more balanced per-class metrics, reported in Section 6.1.2, are read as evidence that this expansion achieved its intended purpose.

8. Limitations

The most significant and clearly documented limitation of the current system is a domain gap between the clinical scanner images that dominate the mega model's training corpus and the images produced by this project's own raw radio frequency reconstruction pipeline. While 880 OASBUD-derived images were deliberately included in training, as described in Section 5.1, specifically to expose the classifier to at least some examples of this kind, the classifier's accuracy drops substantially when evaluated against a held-out set of authentic OASBUD patients reconstructed through the same per-column envelope detection pipeline, falling to approximately 58 percent. This drop is attributed to genuine differences between how clinical scanners and this project's own A-line reconstruction render tissue, rather than to any classification logic error, since the same model continues to perform as reported in Section 6.1.2 on clinical scanner images drawn from the other four datasets. This limitation is stated here plainly rather than minimised, since it represents a meaningful gap between offline benchmark accuracy and real-world performance specifically on this project's own raw-data reconstruction path, and is the clearest direction for future improvement identified in Section 9.

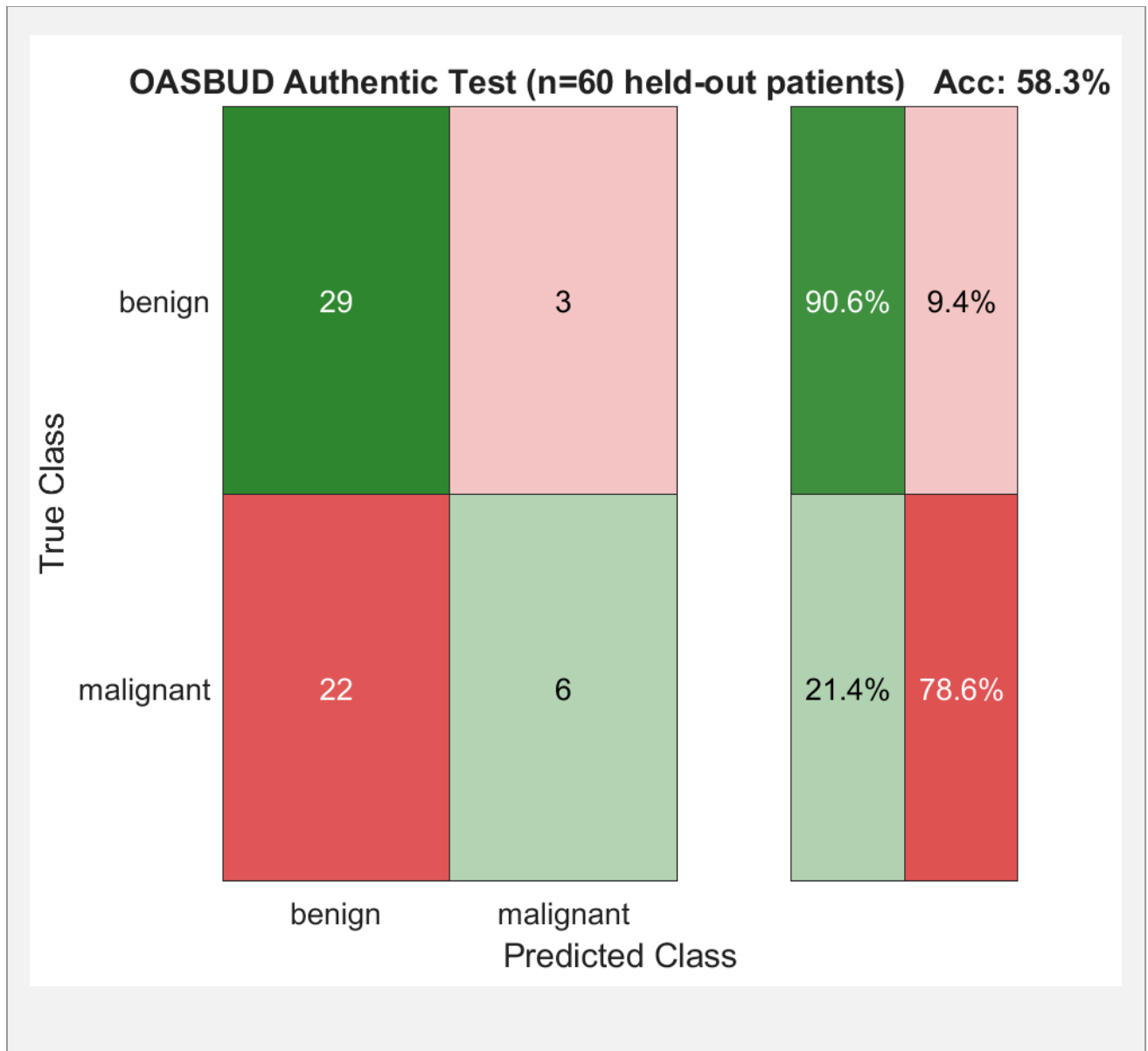


Figure 12. Confusion matrix, MobileNetV2 mega model evaluated against authentic, held-out OASBUD patients reconstructed through the project's own beamforming pipeline, illustrating the domain gap.

9. Future Work

The throughput breakdown in Section 6.2 identifies beamforming as the dominant cost in the current pipeline, accounting for roughly 80 percent of per-frame latency. A CUDA implementation of the beamforming algorithm was already generated from MATLAB using GPU Coder during this project, producing a working CUDA library, but this generated library has not yet been integrated into the Python-based Holoscan beamforming operator, which currently performs the equivalent computation directly in Python. Completing this integration, most likely through a Python ctypes binding to the generated library, is the most direct remaining path to closing the performance gap between the current 34 frames per second and a genuinely real-time target closer to 30 to 60 frames per second with substantial headroom remaining. A second, smaller optimisation opportunity lies in the inference stage, where converting the ONNX model to a TensorRT engine would be expected to reduce the already comparatively small inference latency of 3.49 milliseconds further still, though this represents a much smaller share of total pipeline time than beamforming does. A separate and currently unexploited lever, distinct from accelerating any individual stage, is execution scheduling itself: as Figure 10 makes visible, the current deployment processes frames strictly sequentially, with no overlap between operators across frames, even though Holoscan's underlying scheduler is capable of pipelined, multi-threaded execution where one frame's later stages run concurrently with the next frame's earlier stages. Configuring the pipeline to take advantage of this would be expected to push throughput closer to a bound set by the slowest individual stage rather than by the sum of all stages, independent of and additive to the CUDA and TensorRT optimisations described above.

Narrowing the domain gap described in Section 8 is the most clinically important direction for future work. Possible approaches include reconstructing and adding a substantially larger number of authentic OASBUD-derived images to the training corpus rather than the 880 currently included, exploring domain adaptation techniques explicitly designed to bridge differences between clinical scanner images and reconstruction-derived images, or pursuing higher-fidelity reconstruction methods for raw radio frequency data more broadly. On that last point, the plane-wave compounding investigation described in Section 6.3 suggests a concrete next step: if raw, multi-angle element data of the kind available in the CIRS040GSE phantom dataset, rather than the single pre-formed scan line provided by OASBUD, were available for real patient acquisitions, the same compounding approach validated in that investigation could plausibly produce higher-quality reconstructions of real tissue, which may in turn narrow the domain gap more directly than additional training data alone. This would, in turn, move the project closer to the original challenge brief's suggestion of real Analog Devices analog-to-digital converter hardware integration for genuine raw element acquisition, which remained out of scope for the current submission as stated in Section 2.3.

10. Conclusion

This project set out to build a complete, real-time breast ultrasound classification pipeline, treating real-time deployability and classification accuracy as equally important, connected requirements rather than addressing one in isolation from the other, and to do so through a staged development workflow moving from MATLAB prototyping to Holoscan deployment. Each of the objectives stated in Section 2.3 was met. A beamforming approach correctly matched to the actual structure of the available data was identified and implemented, following an initial misstep that proved instructive in its own right. An image enhancement stage was selected through direct comparative evaluation rather than by default assumption. A classifier was developed through a deliberate, iterative expansion from a single dataset to a five-dataset mega model, with each step in that expansion tied to a specific, diagnosed weakness in the prior stage, reaching an overall test accuracy of approximately 87 percent. The complete pipeline was deployed on NVIDIA Holoscan SDK and measured at a mean throughput of approximately 34 frames per second end to end. A Simulink architecture diagram and a functional enhancement subsystem model were also produced. The functional model simulates the complete image processing subsystem (envelope detection and median filter) using MATLAB Function blocks and was validated against real OASBUD RF data. The domain gap documented in Section 8 remains the clearest unresolved limitation, and is treated throughout this report as an honestly reported finding rather than a deficiency to be obscured, consistent with the project's overall emphasis on measuring and reporting what was actually built and tested rather than what was hoped for.

11. GitHub Repository Guide

11.1 Development History

A private repository was used throughout the building, testing, and debugging process described in this report, serving as the working environment in which the beamforming corrections, enhancement comparisons, progressive model training stages, and Holoscan integration work were all carried out and iterated upon. The public repository accompanying this submission is a separate, finalised result, restructured and populated once the project reached a state ready for formal review, rather than a live mirror of the private development history. The contents and commit history of the private repository are not detailed further here, since they reflect in-progress and superseded work rather than the project's final state.

11.2 Public Repository Structure and Usage

The public submission repository is available at the following address, under the account rohithgreninja-afk.

<https://github.com/rohithgreninja-afk/realtime-ultrasound-holoscan>

It can be obtained by cloning it directly, using git clone followed by the address above, or by downloading it as a zip archive directly from the repository page. The repository root contains this report, the project's README file, and its software licence. A folder named MATLAB Codes contains every MATLAB script described throughout this report, organised by the phase of development each script belongs to, along with the trained mega model in both native MATLAB and ONNX format, and a codegen subfolder containing the CUDA source files generated by GPU Coder from the beamforming algorithm, discussed in Section 9 as a pending integration target. A folder named Holoscan contains the complete set of Python operator implementations described in Section 4.1, together with a standalone benchmarking script used to produce the throughput figures reported in Section 6.2. A folder named Simulink contains both UltrasoundPipelineDiagram.slx (architecture diagram) and UltrasoundEnhancementSubsystem.slx (functional enhancement subsystem, successfully simulated in MATLAB R2024b). A folder named Project Figures contains every figure referenced throughout this report, organised into subfolders by the phase or investigation that produced it, including a dedicated subfolder for the plane-wave compounding study described in Section 6.3. A reader wishing to review the project's reasoning and results without running any code can do so by reading the README and this report directly. A reader wishing to reproduce specific results would need MATLAB R2024b with the Deep Learning Toolbox, Image Processing Toolbox, and GPU Coder, an NVIDIA GPU, and, for the real-time deployment specifically, a WSL2 Ubuntu

environment with the NVIDIA Holoscan SDK installed, with the README providing the specific setup commands required for each environment.

12. References

- Al-Dhabyani, W., Gomaa, M., Khaled, H., and Fahmy, A. (2020). Dataset of Breast Ultrasound Images. *Data in Brief*, 28, 104863.
- Piotrkowska-Wroblewska, H., Dobruch-Sobczak, K., Byra, M., and Nowicki, A. Open access database of raw ultrasonic signals acquired from malignant and benign breast lesions (OASBUD). Zenodo, doi: 10.5281/zenodo.545928.
- BUS-UCLM Breast Ultrasound Lesion Segmentation Dataset, University of Castilla-La Mancha.
- BUS-BRA: A Breast Ultrasound Dataset for Assessing Computer-aided Diagnosis Systems. Zenodo.
- BrEaST: Breast Lesions Ultrasound Dataset. The Cancer Imaging Archive, distributed under a CC-BY 4.0 licence.
- Ultrasound Plane Wave Raw Data, 75 Angles, Breast Phantom and Calibration Phantom Dataset (CIRS040GSE and CIRS073_RUMC). Zenodo, record 7986407.
- Howard, A., Zhmoginov, A., Chen, L.-C., Sandler, M., and Zhu, M. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks.
- MathWorks. MATLAB R2024b Documentation, including the Deep Learning Toolbox, Image Processing Toolbox, GPU Coder, and Phased Array System Toolbox.
- NVIDIA Corporation. NVIDIA Holoscan SDK Documentation, version 4.2.0.

13. Appendix

This appendix collects two short code excerpts referenced in earlier sections, included here for readers who wish to see the exact implementation without consulting the repository directly.

13.1 OASBUD Beamforming, Corrected Approach

The following excerpt implements the per-column Hilbert envelope and power-law compression approach described in Section 5.2, used wherever OASBUD radio frequency data is reconstructed into an image elsewhere in this project.

```
env = abs(hilbert(rfColumn));
env = env ./ max(env(:));
bmode = env .^ 0.3;    % power law compression, gamma = 0.3
```

13.2 Holoscan Inference Operator, Preprocessing Convention

The following excerpt summarises the preprocessing convention established in Section 5.5 and Section 7 for the Holoscan inference operator, reflecting the resolution of the double softmax and double normalisation bugs described there.

```
# Input: raw float32 pixel values, range 0-255, no normalisation applied here
# Output node 'new_softmax' already contains final class probabilities
outputs = session.run(None, {input_name: raw_image_float32})
probabilities = outputs[0][0]    # no softmax applied again
```

14. Generative AI Usage and Acknowledgment

Generative AI (Claude, Anthropic) was used during this project to assist with debugging (ONNX inference normalisation and softmax issues, GPU Coder environment configuration), Holoscan pipeline integration, and drafting of this documentation. All technical decisions, beamforming corrections, model architecture choices, and reported results were reviewed, tested, and verified by the author, who can explain and reproduce all code submitted with this project.

The public GitHub repository accompanying this report bundles a small 10-patient sample (5 malignant, 5 benign) from the OASBUD dataset under `data/sample/`, rather than the full 100-patient set, to keep the repository lightweight (approximately 31 MB) while still providing a class-balanced smoke test for reviewers. Reproducing the full results reported in this document requires the complete OASBUD dataset, downloadable from the source cited in the References section, pointed to via the `OASBUD_PATH` environment variable as described in the repository README.